

```

if (cond1) {
    // some code
    y = x + 4;
} else {
    // other code
    a = x + 4;
}
z = x + 4;

```

In the above code snippet, the computation of the expression ' $(x+4)$ ' in the statement ' $z = x + 4$ ' is fully redundant since this expression is available along all paths reaching this statement. Therefore, the compiler can eliminate this computation. The modified code after common sub-expression elimination is shown below.

```

if (cond1) {
    // some code
    t = x + 4;
    y = t;
} else {
    // other code
    t = x + 4;
    a = t;
}
z = t;

```

However, note that we can apply common sub-expression elimination only if the expression is available along all paths reaching the point P. Now, can you eliminate the computation of $(x + 4)$ in the statement ' $z = x + 4$ ' in the code snippet shown below?

```

if (cond1) {
    // some code
    y = x + 4;
} else {
    // other code
}
z = x + 4;

```

You cannot apply common sub-expression elimination here, since the expression $(x + 4)$ is not available along the path containing the *else*-part of the *if* statement reaching the statement ' $z = x + 4$ '. However, partial redundancy elimination (PRE) can be applied to eliminate this redundant computation.

All the optimisations we have discussed here are known as scalar optimisations. Compilers also support optimisations targeted at loops in the code. These are known as loop optimisations. Recall that applications spend most of their time waiting for data from the main memory. In fact, a study of

enterprise applications shows that more than 60 per cent of application execution time is lost in data cache miss stall cycles. Therefore, the primary aim of loop optimisations is to reduce data cache misses experienced by an application. Some of the popular loop optimisations are loop interchange, loop unswitching, loop tiling, loop fission, loop fusion, loop unrolling and strip mining. We will discuss some of the loop optimisations in next month's column.

Takeaway question for this month

Consider the following code snippet from a C program.

```

for (int j = 0; j < 20; j++) {
    for (int i = 0; i < 10; i++) {
        a[i][j] = i + j;
    }
}

```

Remember that arrays are allocated in row-major form in C applications. This means that the contiguous elements of the row are placed together. How do you rewrite the above code snippet to reduce the data cache misses incurred by this code snippet?

My 'must-read' article for this month

The recommendation for this month's 'must-read' article comes from our reader Abhinav Kumar. 'When algorithms control the world' is an online article by BBC reporter, Jane Wakefield, available at <http://www.bbc.co.uk/news/technology-14306146>. Thanks to Abhinav for the recommendation. I read this article and found it to be thought-provoking. Though it was a bit one-sided, focusing more on the overt impact of algorithms on everyday life, it was an interesting read.

If you have a favourite programming book/article that you think is a must-read for every programmer, please do send me a note with the book's name, and a short write-up on why you think it is useful, so I can mention it in this column. This would help many readers who want to improve their coding skills.

If you have any favourite programming puzzles that you would like to discuss on this forum, please send them to me, along with your solutions and feedback, at sandyasm_AT_yahoo_DOT_com. Till we meet again next month, happy programming, and here's wishing you the very best! 

By: Sandya Mannarswamy

The author is an expert in systems software, and works at Hewlett-Packard India. Her interests include compilers, multi-core technologies and software development tools. If you are preparing for systems software interviews, you may find it useful to visit Sandya's LinkedIn group 'Computer Science Interview Training India' at <http://www.linkedin.com/groups?home=&gid=2339182>.